APR-BALS

Adaptive Per-Tile Precision for Reordered Blocked Alternating Least Squares on GPUs for Recommender Systems

An Undergraduate Honors Thesis Proposal

Submitted by: [Student Name]
Roll No: [XXXX]
Session: [XXXX–XX]
Supervisor: [Supervisor Name]
Department: Computer Science & Engineering
Institution: [University Name], Khulna

April 27, 2026

Abstract

Matrix Factorization (MF) is a foundational algorithm for modern recommender systems, but its scalability on the massive sparse rating matrices of real-world platforms remains a bottleneck. The state-of-the-art **BALS** (Blocked Alternating Least Squares, Chen et al., IEEE TPDS 2021) solver achieves leading GPU performance through 2D-tile blocking and a data-reordering technique that produces a striking spatial pattern: dense tiles concentrate in the top-left of the reordered matrix, while sparse tiles dominate the bottom-right. *This spatial heterogeneity has never been exploited.* Concurrently, mixed-precision arithmetic on GPU Tensor Cores has matured for dense GEMM and SpMV, but no prior work applies *per-tile, density-aware* precision selection to the ALS solver itself. This thesis proposes **APR-BALS**, a method that (i) classifies each BALS tile into one of three precision tiers (FP16, TF32, FP32) based on its non-zero density, (ii) dispatches the corresponding precision kernel for the ALS $Y^\top Y$ accumulation, and (iii) formalizes a density-to-precision mapping rule preserving RMSE convergence. We expect APR-BALS to deliver 1.5–2.5× speedup over BALS on standard MovieLens, Netflix, and YahooMusic benchmarks while preserving recommendation quality, opening a publishable mid-tier conference contribution at the intersection of high-performance computing and recommender systems.

Keywords: Matrix Factorization, Alternating Least Squares, GPU Computing, Mixed Precision, Tensor Cores, Recommender Systems, CUDA.

Contents

1 Introduction

Recommender systems underpin a substantial share of contemporary digital experiences — from movie suggestions on Netflix and product recommendations on Amazon to news ranking on social platforms. At the heart of most of these systems lies **Matrix Factorization (MF)**: the rating matrix $R \in \mathbb{R}^{m \times n}$ is approximated as the product of two low-rank factors $X \in \mathbb{R}^{m \times f}$ and $\Theta \in \mathbb{R}^{n \times f}$, where f is the number of latent features. The two dominant solvers are *Stochastic Gradient Descent* (SGD) and *Alternating Least Squares* (ALS); both have been actively accelerated on GPUs over the last decade.

The challenge is scale. A modern rating matrix can hold billions of non-zeros; even one ALS iteration involves expensive sparse matrix accumulations and dense linear-system solves for every user and every item. GPU implementations such as cuMF_ALS, cuMF_SGD, cu2rec, and BALS have pushed the throughput envelope progressively, with BALS (Chen et al., 2021) currently representing the state of the art for ALS on a single GPU.

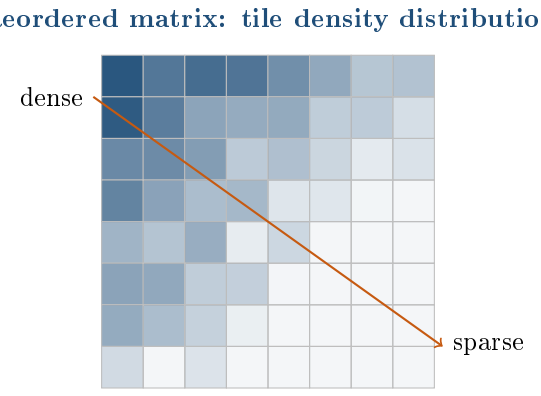
Two recent trends in high-performance computing remain unconnected to this line of work. First, modern NVIDIA GPUs (Volta and newer) include **Tensor Cores** that deliver 4–8× higher throughput on FP16/TF32 arithmetic than on FP32. Second, **mixed-precision techniques** — using lower precision selectively where numerical sensitivity permits — have been shown to cut runtime substantially on dense GEMM, sparse SpMV, and iterative refinement solvers. *No prior work brings these two trends to ALS-based MF, and in particular none exploits the spatial structure that BALS's data reordering produces.*

This proposal lays out a research plan for an undergraduate honors thesis to fill exactly that gap.

2 Motivation and Problem Statement

2.1 The BALS Reordering Phenomenon

BALS reorders rows and columns of the sparse rating matrix by non-zero count, sorting heavy users and popular items toward the top-left. The result, illustrated schematically below, is a matrix with a striking density gradient:



This is a free signal that BALS does not exploit. The original BALS paper uses the same FP32 kernel for every tile regardless of density.

2.2 The Problem

Tile density and the numerical conditioning of the corresponding $Y^\top Y$ accumulation are correlated. Dense tiles aggregate many non-zero contributions, smoothing out the rounding noise

of low-precision arithmetic; sparse tiles, with few contributors per output element, are more sensitive to precision loss. Yet BALS treats them identically.

Research Question

Can per-tile precision dispatch — driven by tile density measured directly from the reordered BALS storage format — yield a measurable end-to-end speedup on standard recommender benchmarks (MovieLens-20M, Netflix, YahooMusic) without harming RMSE convergence?

3 Background

3.1 Alternating Least Squares (ALS)

ALS minimizes the regularized loss

$$\mathcal{L}(X,\Theta) = \sum_{(u,v)\in\Omega} \Big(R_{uv} - X_u^\top \Theta_v\Big)^2 + \lambda\big(\|X\|_F^2 + \|\Theta\|_F^2\big)$$

by alternating two convex sub-problems. Holding Θ fixed, every user row X_u is solved as the closed-form least-squares solution

$$X_u = \Big(\Theta_{\Omega_u}^\top \Theta_{\Omega_u} + \lambda I\Big)^{-1} \Theta_{\Omega_u}^\top R_{u,\Omega_u},$$

where Ω_u is the set of items rated by user u. The $Y^\top Y$ accumulation in this expression is the dominant cost on GPU.

3.2 BALS Storage and Reordering

BALS partitions the sparse matrix into 2D tiles of fixed dimension $T \times T$. Tile metadata (`tile_ptr`, `tile_idx`) lets a CUDA kernel stream tiles into shared memory exactly once per ALS iteration, eliminating the redundant loads that hurt earlier GPU implementations. The non-zero-count reordering of users and items happens once at preprocessing and is essentially free at runtime.

3.3 GPU Mixed-Precision Arithmetic

Modern NVIDIA GPUs offer three useful precisions for our setting:

Format	Range / use	TFLOPS (A100)
FP32	Standard single precision; safest baseline	19.5
TF32	19-bit FP32-range / FP16-precision; Tensor-Core enabled	156
FP16	Half precision; smallest range; risk of overflow	312

The 8–16× throughput gap between FP32 and FP16 is the carrot that motivates this work.

4 Related Work and Research Gap

4.1 The Landscape

Work	Year	Domain	What it does (and does not do)
BALS [1]	2021	ALS MF	2D tile blocking + reordering. *No mixed precision.*
cuMF_ALS approx [2]	2018	ALS MF	FP16 in CG solver *uniformly*; not per-tile, not density-aware.
BDMP [6]	2024	SpMV	Block-wise dynamic mixed precision *for SpMV*, not for MF.
tSparse [7]	2020	spGEMM	Tensor-Core spGEMM. Different operation than ALS $Y^\top Y$.
TCA-SpMM	2024	SpMM	Tensor-Core SpMM, not ALS.
PaRSEC tile-mixed-GEMM [8]	2025	Dense GEMM	Tile-centric mixed precision *for dense* GEMM.
HSGD* [5], UMA-MF, HCC-MF	2021–23	SGD MF	Heterogeneous CPU-GPU SGD; no precision tuning.

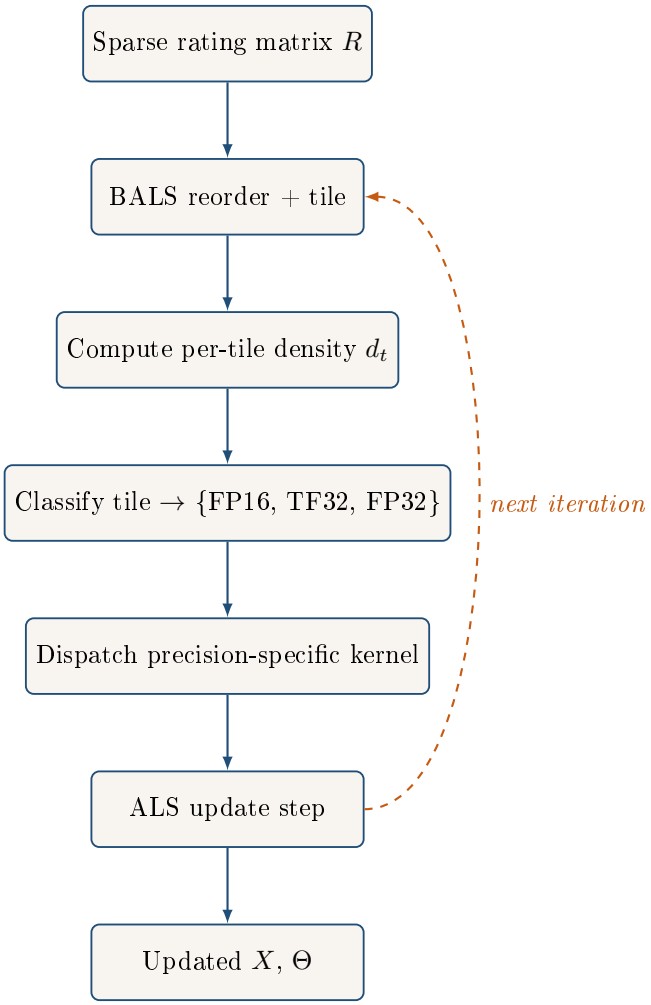
4.2 The Gap

Identified Research Gap

Three technical elements have never been combined: **(i)** exploiting the spatial density gradient produced by BALS reordering, **(ii)** per-tile precision dispatch within the ALS $Y^\top Y$ accumulation, and **(iii)** a principled density-to-precision rule for matrix factorization. Existing mixed-precision tile work targets dense GEMM or SpMV, not ALS. The gap is genuine, narrow, and defensible.

5 Proposed Approach: APR-BALS

5.1 High-Level Design



5.2 Density-to-Precision Mapping

We define a tile's density as $d_t = \mathrm{nnz}(t) \,/\, T^2$ and propose the initial classifier

$$\mathrm{precision}(t) = \begin{cases} \mathrm{FP16} & \text{if } d_t \geq \tau_1, \\ \mathrm{TF32} & \text{if } \tau_2 \leq d_t < \tau_1, \\ \mathrm{FP32} & \text{if } d_t < \tau_2, \end{cases}$$

with thresholds τ_1, τ_2 to be tuned empirically per dataset. The intuition: more contributing terms in dense tiles average out FP16 rounding noise, while sparse tiles cannot rely on this averaging and need higher precision.

5.3 What is Novel

1. **Spatial exploitation:** APR-BALS is the first to use BALS reordering's spatial density structure for an algorithmic purpose beyond memory locality.
2. **Per-tile precision in ALS:** Earlier mixed-precision MF work (cuMF_ALS) used uniform FP16; APR-BALS adapts at the tile granularity.
3. **Density-driven classifier:** A formal rule mapping local non-zero density to numerical precision tier, with empirical thresholds calibrated per dataset.

6 Methodology

6.1 Implementation Plan

1. Clone and build the BALS reference implementation (or, as fallback, reproduce its 2D tile blocking on top of cuMF_ALS).
2. Profile baseline BALS on MovieLens-100k with `nsight-compute` to identify the bottleneck kernels.
3. Implement three precision-specialized variants of the $Y^\top Y$ kernel using `cuBLAS` half-precision routines and `wmma::` Tensor Core intrinsics.
4. Implement the per-tile density classifier and dispatcher.
5. Calibrate τ_1, τ_2 on small datasets, then validate on full benchmarks.
6. Compare against BALS, cuMF_ALS, and cuMF_SGD on speed and RMSE.

6.2 Datasets

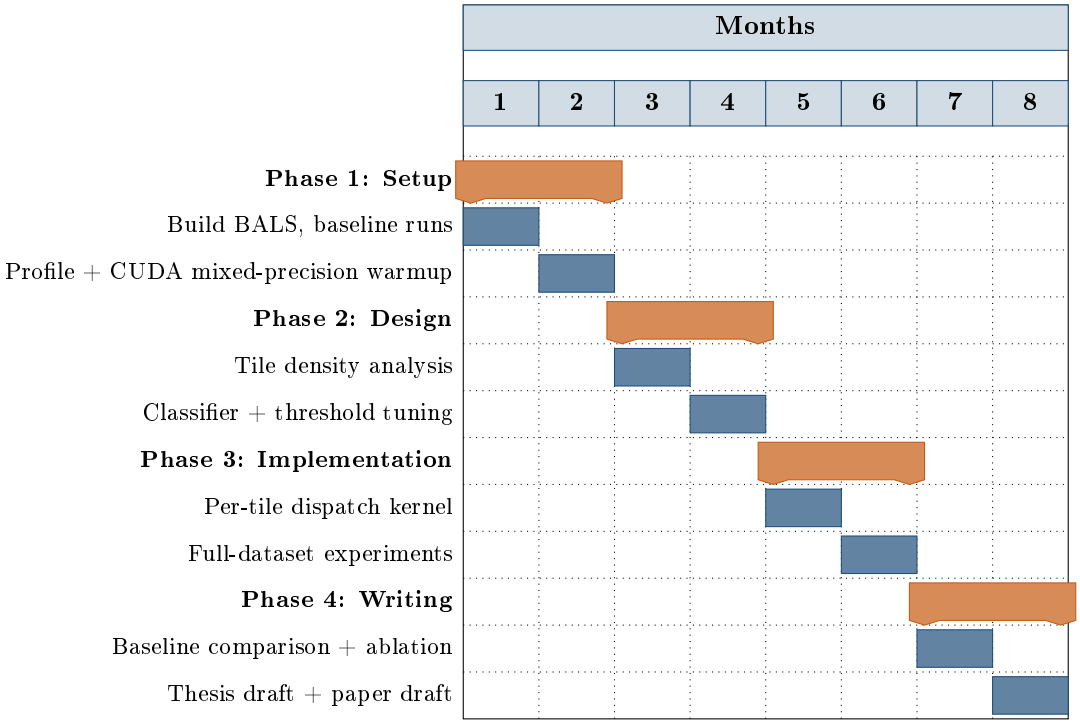
Dataset	Users	Items	Ratings
MovieLens-100k	943	1,682	100,000
MovieLens-1M	6,040	3,706	1,000,209
MovieLens-20M	138,493	27,278	20,000,263
Netflix Prize	480,189	17,770	100,480,507
YahooMusic R1	1,948,882	98,213	256,804,235

6.3 Evaluation Metrics

- **Speedup:** per-iteration and end-to-end vs. BALS, cuMF_ALS.
- **Quality:** test-set RMSE; convergence curves over iterations.
- **Memory bandwidth utilization:** measured with NVIDIA profiling tools.
- **Precision-tier distribution:** fraction of tiles falling into each tier per dataset.

7 Project Timeline

The thesis is planned over an 8-month timeline, organized into four 2-month phases.



7.1 Detailed Milestones

Month	Phase	Deliverable
1–2	Setup & Profiling	BALS running on MovieLens-100k; profile report; FP16 GEMM toy example.
3–4	Design	Tile-density distribution plots; precision classifier with calibrated thresholds.
5–6	Implementation	Working APR-BALS kernel; experiments on MovieLens-1M / 20M, Netflix, YahooMusic.
7–8	Analysis & Writing	Comparison with BALS / cuMF_ALS; ablation study; thesis manuscript and conference paper draft.

8 Risks and Mitigation

Risk	Severity	Mitigation
BALS source code unavailable	Medium	Fall back on cuMF_ALS; reimplement 2D tile blocking (1 month overhead).
GPU lacks Tensor Cores (Pascal or older)	Medium	FP16 still works without Tensor Cores; benefit reduces from $\sim 8\times$ to $\sim 2\times$ but contribution remains valid.
FP16 hurts RMSE on sparse tiles	Low	The classifier already routes sparse tiles to FP32; threshold tuning protects convergence.
PaRSEC publishes ALS extension first	Low	Their work is on dense GEMM; ALS-specific contribution remains distinct.
Insufficient dataset memory	Low	Stage to disk and stream; MovieLens-20M fits in 4–8 GB GPU memory comfortably.

9 Expected Contributions

1. **An algorithm:** APR-BALS, the first per-tile adaptive-precision variant of blocked ALS for sparse matrix factorization on GPUs.
2. **A classifier:** a calibrated density-to-precision mapping rule for the ALS $Y^\top Y$ accumulation.
3. **An empirical study:** a comparison on five standard recommender benchmarks against BALS, cuMF_ALS, and cuMF_SGD measuring speed, RMSE, and memory-bandwidth utilization.
4. **Open-source release:** CUDA implementation released on GitHub for reproducibility.
5. **Publication target:** a 6–10-page paper at a mid-tier HPC venue such as ICPP, IPDPS, HPDC, or a recommender-systems venue such as RecSys's HPC track.

10 Tools and Resources

- **Languages:** CUDA C++, Python (data preprocessing, plotting).
- **Libraries:** cuBLAS, cuSPARSE, NVIDIA `wmma::` API, Thrust.
- **Profilers:** `nsight-compute`, `nsight-systems`.
- **Hardware target:** any single NVIDIA GPU with compute capability ≥ 7.0 (Volta+); Tensor Cores preferred.
- **Baselines:** BALS source, cuMF_ALS (`github.com/cuMF/cumf_als`), cuMF_SGD.

References

References

[1] Y. Chen, J. Fang, F. Liu, et al., "BALS: Blocked Alternating Least Squares for Parallel Sparse Matrix Factorization on GPUs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 9, pp. 2291–2302, 2021.

[2] W. Tan, S. Cao, L. L. Fong, "Matrix Factorization on GPUs with Memory Optimization and Approximate Computing," *Proc. ICPP*, 2018, pp. 26–35.

[3] X. Xie, W. Tan, L. L. Fong, Y. Liang, "CuMF_SGD: Parallelized Stochastic Gradient Descent for Matrix Factorization on GPUs," *Proc. HPDC*, 2017.

[4] N. Greenquist, B. Mitra, D. Kilitcioglu, "cu2rec: GPU-Accelerated Matrix Factorization for Recommender Systems," NYU Course Project Report, 2018.

[5] H. Yu, X. Bai, J. Wang, et al., "Efficient Matrix Factorization on Heterogeneous CPU-GPU Systems," *Proc. ICDE*, 2021.

[6] J. Zhao, Y. Wen, et al., "Block-wise Dynamic Mixed-Precision for Sparse Matrix-Vector Multiplication on GPUs," *The Journal of Supercomputing*, 80(13), 2024.

[7] O. Zachariadis, N. Satpute, J. Gómez-Luna, J. Olivares, "Accelerating Sparse Matrix-Matrix Multiplication with GPU Tensor Cores," *Computers & Electrical Engineering*, vol. 88, 2020.

[8] Q. Cao, R. Alomairy, et al., "Leveraging Hardware-Aware Computation in Mixed-Precision Matrix Multiply: A Tile-Centric Approach," *Proc. ISC High Performance*, 2025.

[9] A. Haidar, S. Tomov, J. Dongarra, N. J. Higham, "Harnessing GPU Tensor Cores for Fast FP16 Arithmetic to Speed up Mixed-Precision Iterative Refinement Solvers," *Proc. SC18*, 2018.

[10] Y. Koren, R. Bell, C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *IEEE Computer*, vol. 42, no. 8, pp. 30–37, 2009.

End of Proposal